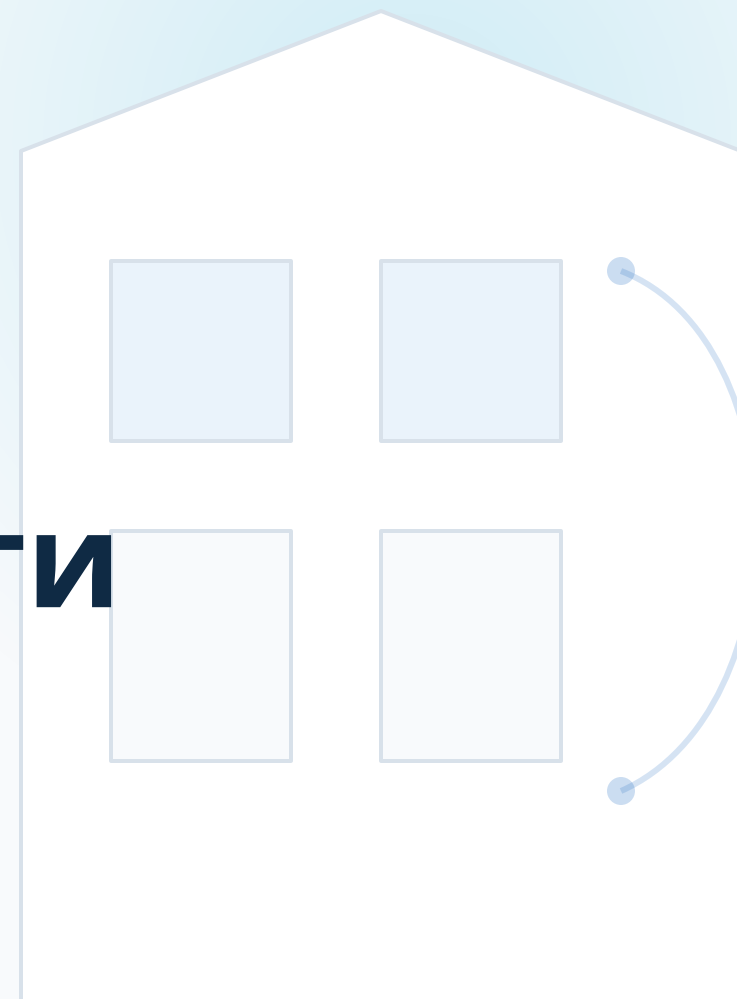


FINAL DEFENSE

Классификация типов помещений по фото недвижимости

Воспроизводимое ML-решение для модерации,
поиска и аналитики объявлений



Ручная проверка типов помещений замедляет модерацию и ухудшает качество данных

Бизнес-проблема



Тип помещения влияет на клиентский опыт

Ошибки в разметке ломают фильтры, поиск, рекомендации и аналитику.



Фото неоднородны

Качество, ракурс, освещение и мебель сильно меняют визуальный сигнал.



Классы похожи друг на друга

Спальня, гостиная и универсальная комната требуют вероятностного ответа.

Мы собрали полный путь от сырых фото до проверенного файла отправки и демо



Схема задачи

Классификация фото недвижимости

20 классов

от кухни до “не ясно” и “комната без мебели”



Главный результат

Macro F1 не скрывает редкие классы

0.7934 $\approx$ OOF Macro F1

исследовательский кандидат: ConvNeXt XL + Focal Loss



Воспроизводимость

Пайплайн запускается на новом окружении

train → **infer**

``make full-pipeline`, `make infer`, `validate_submission``



Готовность к демонстрации

Код, конфиг, веса, Docker, демо и CSV

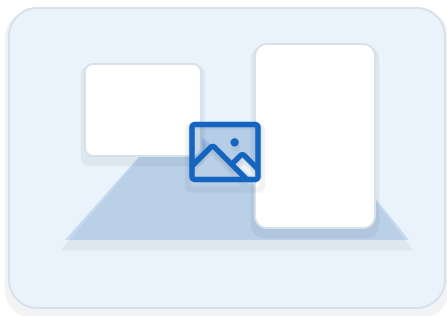
rc1.yaml release config

одна точка входа для весов, mapping и предсказаний

Модель возвращает не только класс, но и уверенность вместе с top-k альтернативами

Такой формат ответа безопасен для бизнеса: автоматизация применяется только там, где уверенность достаточно высокая

Вход



Одно фото объявления

Разное качество, ракурс, освещение и меблировка.



RoomPredictor

модель классификации

20 классов · top-k



Выход для продукта

```
{
  "class_id"    : 3,
  "class_name"  : "гостиная",
  "probability" : 0.87,
  "top_k"       : [3, 1, 2]
}
```

уверенно



ускорить модерацию

Качество решения начинается с защиты от грязных данных и утечек

Без этого высокий скор на validation мог бы быть иллюзией, а не сигналом готовности к реальному применению



1. Аудит файлов

Что проверяли

наличие изображений
открываемость файлов
таблица путей

`data/preprocess`



2. Дубли

Как контролировали

расчёт content_hash
поиск одинаковых фото
контроль повторов объектов

`content_hash`



3. Split без утечки

Правило качества

5 частей обучения
отдельная проверка
одинаковые объекты не смешиваем

`5-fold + holdout`



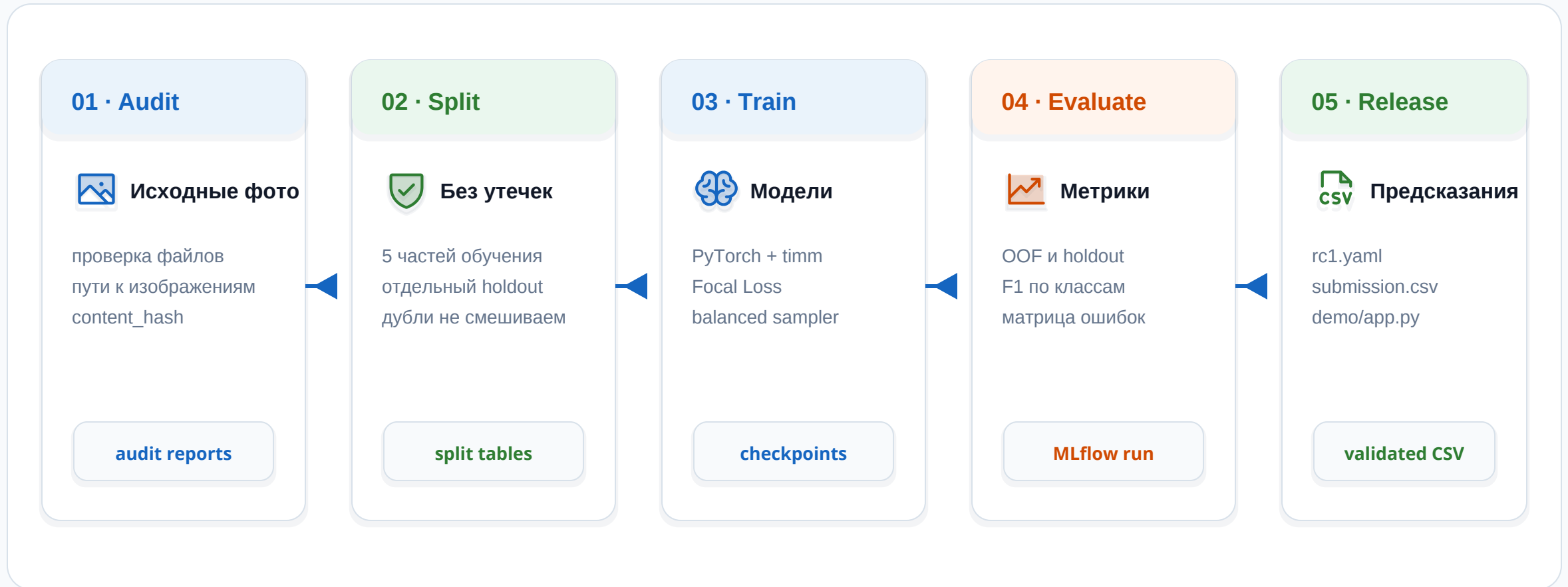
4. Метрика

Почему Macro F1

не прячет редкие классы
F1 по каждому классу
матрица ошибок

`class-level audit`

Процесс воспроизводим: каждый шаг оставляет проверяемый артефакт



Мы проверили архитектуры и способы борьбы с дисбалансом, затем зафиксировали release-кандидата

Эксперименты были не перебором ради перебора: каждый шаг закрывал конкретный риск качества

Baseline

01

Backbones

02

Imbalance

03

Validation

04

Release

05

Стартовая линия

простой baseline
быстрая smoke-проверка
единая метрика

baseline run

Архитектуры

ConvNeXt, EfficientNet
ResNet
ViT / CLIP

timm configs

Дисбаланс

взвешенная loss
Focal Loss
balanced sampler

rare classes

Проверки

OOF + holdout
F1 по классам
матрица ошибок

run_id only

Фиксация

release_cv03
rc1.yaml
checkpoint + CSV

release ready

Итоговый research-кандидат: ConvNeXt XL, 512×512, Focal Loss, balanced sampler, AMP, early stopping

Macro F1 ≈ 0.7934 показывает, что качество не маскируется частыми классами



Основные риски — похожие жилые комнаты и редкие классы; их закрывают top-k и ручная проверка

Зоны визуальной похожести классов

Интенсивность — где модель чаще нуждается в top-k и модераторе

	спальня	гостиная	универс.	кух.-гост.	санузел
спальня	НИЗК.	ВЫС.	ВЫС.	сред.	НИЗК.
гостиная	ВЫС.	НИЗК.	ВЫС.	сред.	НИЗК.
универс.	ВЫС.	ВЫС.	НИЗК.	сред.	НИЗК.
кух.-гост.	сред.	сред.	сред.	НИЗК.	НИЗК.
санузел	НИЗК.	НИЗК.	НИЗК.	НИЗК.	НИЗК.

Что делаем с риском



Показываем top-k

модератор видит альтернативы, а не только один класс



Порог уверенности

низкая вероятность уходит на ручную проверку



Class-level контроль

не ограничиваемся средней метрикой по датасету

Решение можно повторно запустить локально, через Docker или через release-конфиг



Data layer

`data/raw/`, проверка файлов, `content\_hash`, schema audit, class mapping



Training & evaluation layer

PyTorch + timm, AMP, early stopping, Focal Loss, MLflow tracking, OOF и holdout



Release & inference layer

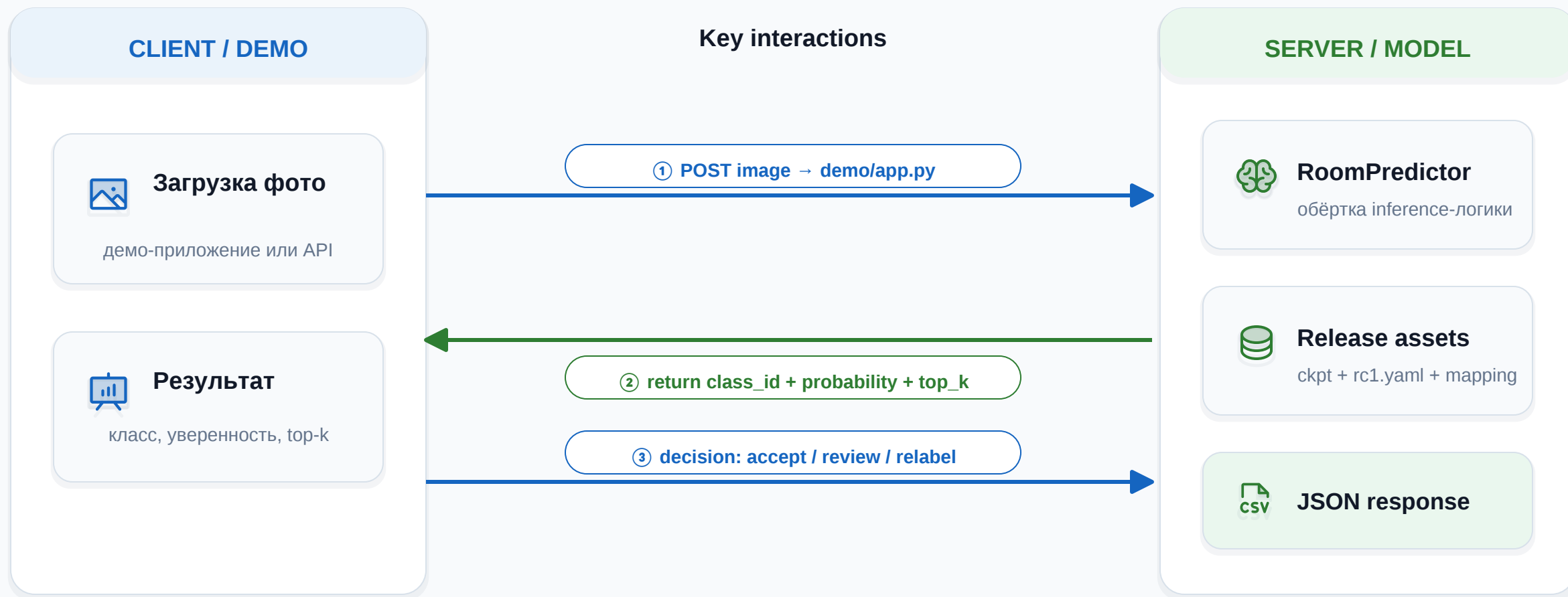
`configs/release/rc1.yaml`, checkpoint, `RoomPredictor`, `submission.csv`, validation before sending

Docker

Jupyter

demo/app.py

Демо показывает путь интеграции: фото → RoomPredictor → JSON-ответ → интерфейс модерации



Все результаты и ход работы задокументированы: код, архитектура, MLflow, правила и демо



Текстовый документ

архитектура и ход работы

[docs/ARCHITECTURE.md](#)



Журнал опытов

метрики, параметры, run_id

[docs/ML_FLOW.md](#)



Правила проекта

PR, проверки, зоны ответственности

[docs/project-rules.md](#)



Демо и release

запуск демонстрации и inference

[demo/app.py · rc1.yaml](#)



Файл отправки

ровно `image\_id\_ext,Predicted`

[releases/rc1/submission.csv](#)



Доп. материалы

таблицы, Figma, репозиторий

[вставить финальные URL](#)

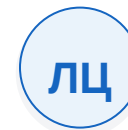
Команда закрыла полный цикл: данные, модели, качество, inference, документация и защита



Давид Сухашвили
руководитель, модель, защита



Алексей Тепляков
данные и первичный анализ



Лина Цаплева
опыты с моделями



Дарья Вдовина
оценка качества



Михаил Калмыков
inference, сервер, Docker



Глеб Медведев
документация и демо